

Lecture 6: Data Acquisition

James Sears*

AFRE 891/991 FS 25

Michigan State University

*Parts of these slides are adapted from **“Data Science for Economists”** by Grant McDermott and **“Advanced Data Analytics”** by Nick Hagerty.

Table of Contents

1. **Prologue**
2. **Intro to Web Scraping**
3. **Scraping Static Websites - Part 1**

Prologue

Prologue

Let's load in the packages that we'll need for today.

```
pacman::p_load(lubridate, rvest, stringr, tidyverse)
```


Data Acquisition

In order to wrangle, clean, or visualize data, we first need... data.



Where Do Data Come from?

There is a whole spectrum, from DIY to plug-and-chug.

1. Pre-cleaned datasets posted on secondary repositories

- General and journal replication repositories
 - e.g. [Harvard Dataverse](#)
- Community-based repositories
 - e.g. [Kaggle](#)
- Public GitHub repositories
 - e.g. [Johns Hopkins COVID-19 caseloads, deaths, and vaccination data](#)

Where Do Data Come from?

There is a whole spectrum, from DIY to plug-and-chug.

2. Open data libraries

- US Gov't: [Data.gov](https://data.gov)
- Institutions, non-profits, and thinktanks
 - e.g. [World Bank Open Data](https://data.worldbank.org), [Pew Research](https://www.pewresearch.org), [NBER Public Use Data](https://www.nber.org/public-uses), [Economic Policy Institute](https://www.epi.org/data), [AEA Data Sources](https://www.eia.gov/data)
- Tech and other sites
 - e.g. data.world, [Our World in Data](https://www.ourworldindata.org), [Stanford Large Network Datasets](https://stanfordlarge.network/datasets)
- [Map of 2600+ Open Data Portals Worldwide](#)

Where Do Data Come from?

There is a whole spectrum, from DIY to plug-and-chug.

3. Websites of primary data providers

- Government agencies; some private companies and NGOs; scientific researchers.
 - i.e. [EPA + USGS Water Quality Portal](#)

4. Programmatic data access through Application Program Interfaces (APIs)

- We'll talk more about these next lecture

Where Do Data Come from?

There is a whole spectrum, from DIY to plug-and-chug.

5. Liberate previously inaccessible data

- Build relationships with people in government or the private sector.
- Find the right person, cold-email them and ask politely.
 - i.e. my master's thesis!
- File a Freedom of Information Act (FOIA) request (a last resort; very aggressive move).

Where Do Data Come from?

6. Compile data yourself

- Assemble systematic information from many disparate sources.
- E.g. historical archives, websites, PDF reports.

7. Collect your own primary data

- Run surveys or experiments.

Where to Look for Data?

There is **no "one-stop shop"**. Where to look entirely depends on your topic.

For economics research:

- **Search the literature:** Find papers related to your topic and check the Data section.
 - Good for learning the "standard" sources for common things (e.g., weather data).
- **Find your own data** that the literature hasn't used yet.
 - It's hard to find a novel use for an already widely-used dataset.
 - Cross-disciplinary arbitrage
- **Combine data in new ways:** most new projects will require joining data from 2+ sources.
 - E.g. state-level policy changes + household-level outcome data.

Where to Look for Data?

There is **no "one-stop shop"**. Where to look entirely depends on your topic.

A few useful starting points:

- ["How to Find Data: Tips for Finding Data"](#) (Davidson College Library).
- ["Data Sets for Quantitative Research"](#) (University of Missouri Libraries).
- [Google Dataset Search](#), [AWS Data Exchange](#)

The majority of data sources described above have the data easily accessible once found.

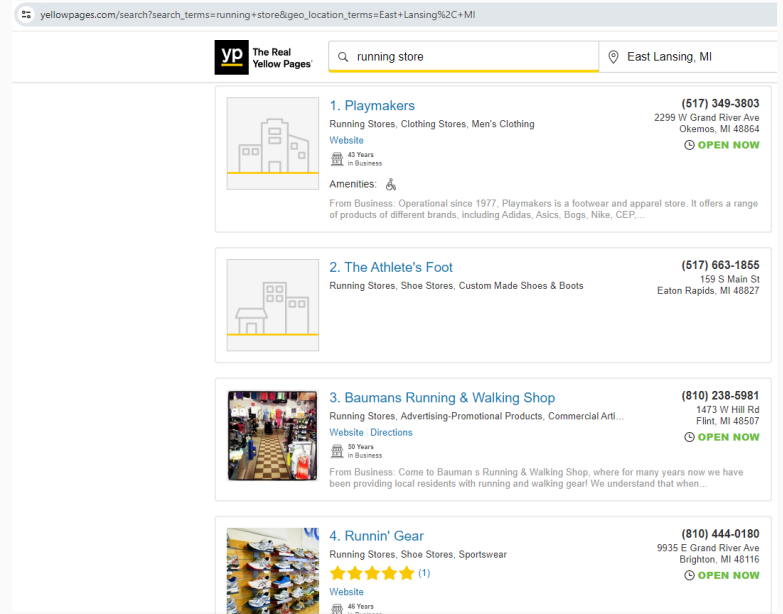
If the data aren't already machine readable, then we can take advantage of...

Intro to Web Scraping

Web Scraping

Web scraping is the process of **extracting semi-structured web data** and **converting into a structured dataset**

- Useful when information is already online but not available in a handy format.



yp The Real Yellow Pages

running store

East Lansing, MI

1. **Playmakers**
Running Stores, Clothing Stores, Men's Clothing
Website
43 Years in Business
Amenities: ♿
From Business: Operational since 1977, Playmakers is a footwear and apparel store. It offers a range of products of different brands, including Adidas, Asics, Bogs, Nike, CEP, ...
(517) 349-3803
2299 W Grand River Ave
Okemos, MI 48864
OPEN NOW

2. **The Athlete's Foot**
Running Stores, Shoe Stores, Custom Made Shoes & Boots
159 S Main St
Eaton Rapids, MI 48827
(517) 663-1855

3. **Baumanns Running & Walking Shop**
Running Stores, Advertising-Promotional Products, Commercial Arti...
Website Directions
50 Years in Business
From Business: Come to Bauman's Running & Walking Shop, where for many years now we have been providing local residents with running and walking gear! We understand that when...
(810) 238-5981
1473 W Hill Rd
Flint, MI 48507
OPEN NOW

4. **Runnin' Gear**
Running Stores, Shoe Stores, Sportswear
★★★★★ (1)
Website
46 Years in Business
(810) 444-0180
9935 E Grand River Ave
Brighton, MI 48116
OPEN NOW

Structure of Webpages

Webpages are largely made out of **two types of files** that we have to parse:

HTML

- HyperText Markup Language is a **markup language**, Like Markdown
- It specifies the **structure** of a webpage.

CSS

- Cascading Style Sheets is a language for **formatting the appearance of a webpage**
 - CSS **properties** specify **how** to format: what font, what color, how wide
 - CSS **selectors** specify **what** to format: which structural elements get what rule

How Websites Render Content

It's also worth realising that there are **two ways** that web content gets rendered in a browser:

1. **Server-side (back-end)**
2. **Client-side (front-end)**

You can read [here](#) for more details (including example scripts), but for our purposes the essential features are as follows...

Server-Side Content Rendering

The scripts that build a **server-side** website aren't run on our computer

- Rather, the script that "builds" the site is **run on a host server**
 - All information is directly embedded in the webpage's HTML
- For example, **Wikipedia** tables are already populated with all the info we see in our browser

No. ♦	Peak ♦	Range (or island) ♦	Location ♦	Coordinates [1] ♦	Prominence (m) ♦	Height (m) ♦	Col (m) ♦
1.	Mount Everest	Himalayas	 China  Nepal	 27°59'17.6500°N 86°55'30.0652°E	8,848.86	8,848.86	0
2.	Aconcagua	Andes	 Argentina	 32°39'11"S 70°0'42"W	6,960.8	6,960.8	0
3.	Denali / Mount McKinley*	Alaska Range	 United States	 63°4'10"N 151°0'26"W	6,155	6,191	47
4.	Mount Kilimanjaro*	Eastern Rift mountains	 Tanzania	 3°4'0"S 37°21'33"E	5,885	5,895	10
5.	Pico Cristóbal Colón	Sierra Nevada de Santa Marta	 Colombia	 10°50'18"N 73°41'12"W	5,509	5,700	191

Server-Side Content Rendering

The scripts that build a **server-side** website aren't run on our computer

- Rather, the script that "builds" the site is **run on a host server**
 - All information is directly embedded in the webpage's HTML
- For example, **Wikipedia** tables are already populated with all the info we see in our browser

- **Webscraping process:** finding the correct selectors (CSS or Xpath), iterating through (dynamic) webpages (e.g. "Next page" and "Show more" tabs)
- **Key concepts:** CSS, Xpath, HTML

Client-Side Content Rendering

The scripts that build a **client-side** website aren't run on our computer

- Website contains an empty template of HTML and CSS
 - May contain a "skeleton" table without any values
- When we visit the page URL, our browser sends a **request** to the host server
- If everything is okay (e.g. our request is valid), then the server sends a **response** script, which our browser executes and uses to populate the HTML template with the specific information that we want.

- If everything is okay (e.g. our request is valid), then the server sends a **response** script, which our browser executes and uses to populate the HTML template with the specific information that we want.

20 / 89

Web Scraping

Over the next two lectures we'll cover the main differences between the two approaches and general workflows.

However, I want to forewarn that web scraping typically involves a fair bit of **detective work**, iterating and adjusting steps

- According to the type of data you want
- To match the specifics of a given website

In short, web scraping involves **as much art as it does science.**

The good news, though: **if you can see it, you can scrape it.**

Ethical + Legal Considerations

The last line brings up an important consideration: just because you **can** scrape it, doesn't mean you **should**.

Legality

- In short, it is **currently legal** to scrape data from the web using automated tools as long as the data are **publicly available** (hiQ Labs vs. LinkedIn)
 - We'll chat more later on about what this means for "hidden" APIs
- May get blocked due to violating a site's Terms of Service preventing scraping

Ethicality

- Need to consider the impact your scraper will have on the host server
 - Easy to write a function that can overwhelm a website's host with rapid requests
 - We'll return to the "be nice" mantra later on

Scraping Static Websites

Static Scraping: Preliminaries

Today we'll be using [SelectorGadget](#), which is a Chrome extension that makes it easy to discover CSS selectors.

- Install the extension directly [here](#).

Please note that SelectorGadget is only available for **Chrome**. If you prefer using **Firefox**, then you can try [ScrapeMate](#).

Static Scraping: Preliminaries

The primary R package that we'll be using today is [rvest](#), a simple web scraping library inspired by Python's [Beautiful Soup](#), but with extra tidyverse functionality.

rvest is designed to work with webpages that are built server-side and thus requires knowledge of the relevant CSS selectors...

Which means that now is probably a good time for us to cover what these are in more detail.

CSS Selectors and SelectorGadget

CSS **selectors** are the **"what"** of the display rules. They identify which rules should be applied to which elements.

- E.g. Text elements that are selected as **".h1"** (i.e. top line headers) are usually larger and displayed more prominently than text elements selected as **".h2"** (i.e. sub-headers).

The key point is that if you can **identify the CSS selector(s)** of the content you want, then you can **isolate it from the rest** of the webpage content that you don't want.

CSS Selectors and SelectorGadget

This where SelectorGadget comes in. We'll work through an extended example (with a twist!) below, but I highly recommend looking over this [quick vignette](#) soon.

Here are two helpful links if you're interested in reading more about [CSS](#) (i.e Cascading Style Sheets) and [SelectorGadget](#)

Application 1: Wikipedia

Let's say you watched the U.S. Olympic Marathon Trials last year and now want to scrape the wikipedia page on [marathon world record progression](#)

- Women's record: 3:40:22 in 1926 to 2:11:53 last year!

First, open up this page in your browser. Take a look at its structure: What type of objects does it contain? How many tables does it have? Do these tables all share the same columns? What row- and columns-spans? Etc.

Application 1: Wikipedia

Contents

hide

(Top)

Criteria for record eligibility

Women's world record

Unofficial record attempts

History

Men

Women

Gallery of world record holders

See also

Notes

References

Sources

External links

Marathon world record progression

ArticleTalk

From Wikipedia, the free encyclopedia

This list is a chronological progression of record times for the marathon. *World records in the marathon* are ratified by *World Athletics*, the international governing body for the sport of athletics.

Kenyan athlete *Kevin Kiptum* set a men's world record time of 2:00:35 on October 8, 2023, at the *2023 Chicago Marathon*.^[1]

Ethiopian athlete *Tigst Assefa* broke the women's world record for a mixed-gender race with a time of 2:11:53 on September 24, 2023, at the *2023 Berlin Marathon*.^[2] In addition to the standard women's marathon world record, World Athletics also recognizes a second world record for women in the "Women Only" category, meaning that the marathon was run on a course without any male athletes in competition. The current "Women Only" record of 2:17:01 was set by *Mary Keitany* on April 23, 2017, at the London Marathon in the elite women's race.^[3]

Criteria for record eligibility

See also: *List of world records in athletics § Criteria*

For a performance to be ratified as a world record by World Athletics, the marathon course on which the performance occurred must be 42.195 km (26.219 mi) long,^[4] measured in a defined manner using the *calibrated bicycle method*^[5] (the distance in kilometers being the official distance, the distance in miles is an approximation) and meet other criteria that rule out artificially fast times produced on courses aided by downhill slope or tailwind.^[6] The criteria include:

- "The start and finish points of a course, measured along a theoretical straight line between them, shall not be further apart than 50% of the race distance."^[4]
- "The decrease in elevation between the start and finish shall not exceed an average of one in a thousand, i.e. 1m per km."^[4]

In recognizing Kenyan *Geoffrey Mutai's* mark of 2:03:02 at the *2011 Boston Marathon* as (at the time) "the fastest Marathon ever run", the IAAF said: "Due to the elevation drop and point-to-point measurements of the Boston course, performances [on that course] are not eligible for World record consideration."^[6]

Road racing events like the marathon were specifically excepted from World Athletics rule 260.18(d) that rejected from consideration those track and field performances set in mixed competition.^[6]

The *Association of Road Racing Statisticians*, an independent organization that compiles data from *road running* events, also maintains an alternate marathon world best progression but with standards they consider to be more stringent.^{[10][11]}

Women's world record

See also: *List of world records in athletics § Criteria*

The IAAF Congress at 2011 World Championships in Athletics passed a motion changing the record eligibility criteria effective October 6, 2007, so that women's world records must be set in all-women competitions.^[12] The result of the change was that Radcliffe's 2:17:42 performance at the 2005 London Marathon would supplant her own existing women's mark as the "world record"; the earlier performance was to be referred to as a "world best".^[12]

The decision was met with strong protest in Britain. In November 2011, an IAAF council member announced that Radcliffe's original mark would be allowed to stand, with the eventual decision that both marks would be



Kevin Kiptum during his world record run at the 2023 Chicago marathon with 2:00:35



Tigst Assefa during her women's

Time	Name	Nationality	Date	Event/Place	Source	Notes
5:40:xx	Marie-Louise Ledru	 France	September 29, 1918	Tour de Paris Marathon	ARRS ^[10]	
3:40:22	Violet Piercy	 United Kingdom	October 3, 1926	London ^[nb 7]	IAAF ^[53]	The ARRS indicates that Piercy's 3:40:22 was set on August 2, 1926, during a time trial on a course that was only 35.4 km ^[10]
3:37:07	Merry Lepper	 United States	December 16, 1963 ^[nb 8]	Culver City, United States	IAAF ^[53]	Disputed (short course) ^[95]
3:27:45	Dale Greig	 United Kingdom	May 23, 1964	Ryde	IAAF ^[53] ARRS ^[10]	
3:19:33	Mildred Sampson	 New Zealand	July 21, 1964 ^[nb 8]	Auckland, New Zealand	IAAF ^[53]	Disputed by ARRS as a time trial ^{[nb 9][96]}
3:14:23	Maureen Wilton	 Canada	May 6, 1967	Toronto, Canada	IAAF ^[53] ARRS ^[10]	The ARRS notes Wilton's extended time as 3:14:22.8 ^[10]
3:07:27.2	Anni Pedersen-Erdkamp	 West Germany	September 16, 1967	Waldniel, West Germany	IAAF ^[53] ARRS ^[10]	The ARRS notes Pedersen-Erdkamp's extended time as 3:07:26.2 ^[10]
3:02:53	Caroline Walker	 United States	February 28, 1970	Seaside, OR	IAAF ^[53] ARRS ^[10]	
3:01:42	Elizabeth Bonner	 United States	May 9, 1971	Philadelphia, United States	IAAF ^[53] ARRS ^[10]	
2:55:22	Elizabeth Bonner	 United States	September 19, 1971	New York City Marathon	IAAF ^[53] ARRS ^[10]	
2:49:40	Cheryl Bridges	 United States	December 5, 1971	Culver City, United States	IAAF ^[53] ARRS ^[10]	
2:46:36	Michiko Gorman	 United States	December 2, 1973	Culver City, United States	IAAF ^[53] ARRS ^[10]	The ARRS notes Gorman's extended time as 2:46:37 ^[10]
2:46:24	Chantal Langlacé	 France	October 27, 1974	Neuf-Brisach, France	IAAF ^[53] ARRS ^[10]	
2:43:54.5	Jacqueline Hansen	 United States	December 1, 1974	Culver City, United States	IAAF ^[53] ARRS ^[10]	The ARRS notes Hansen's extended time as 2:43:54.6 ^[10]

Application 1: Wikipedia

Once you've familiarised yourself with the structure, read the whole page into R using the `rvest::read_html()` function.

```
mthn =  
read_html("https://en.wikipedia.org/wiki/Marathon_world_record_progression")  
mthn
```

```
## {html_document}  
## <html class="client-nojs vector-feature-language-in-header-enabled  
vector-feature-language-in-main-page-header-disabled vector-feature-page-  
tools-pinned-disabled vector-feature-toc-pinned-clientpref-1 vector-  
feature-main-menu-pinned-disabled vector-feature-limited-width-clientpref-  
1 vector-feature-limited-width-content-enabled vector-feature-custom-font-  
size-clientpref-1 vector-feature-appearance-pinned-clientpref-1 vector-  
feature-night-mode-enabled skin-theme-clientpref-day vector-sticky-header-  
enabled vector-toc-available" lang="en" dir="ltr">  
## [1] <head>\n<meta http-equiv="Content-Type" content="text/html;  
charset=UTF-8 ...  
## [2] <body class="skin--responsive skin-vector skin-vector-search-vue  
mediawik ...
```

Application 1: Wikipedia

As you can see, this is an XML document that contains **everything** needed to render the Wikipedia page.¹

It's kind of like viewing someone's entire LaTeX document (preamble, syntax, etc.) when all we want are the data from some tables in their paper.

¹ XML stands for Extensible Markup Language and is one of the primary languages used for encoding and formatting web pages.

First Table: Women's Records

Let's start by scraping our first table from the page, which documents the women's record progression.

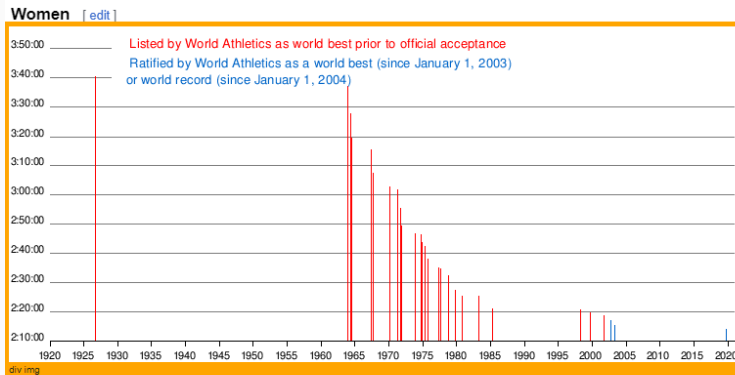
The first thing we need to do is identify the table's unique CSS selector using SelectorGadget.

Note: this *will* require trial and error - and a lot of clicking.

First Table: Women's Records

Start by activating SelectorGadget and clicking on a chart element.

2:01:09	Eliud Kipchoge	 Kenya	September 25, 2022	Berlin Marathon	World Athletics ^[90]	
2:00:35	Kelvin Kiptum	 Kenya	October 8, 2023	Chicago Marathon	World Athletics ^[91]	First man to break 2:01:00 in a record-eligible marathon.



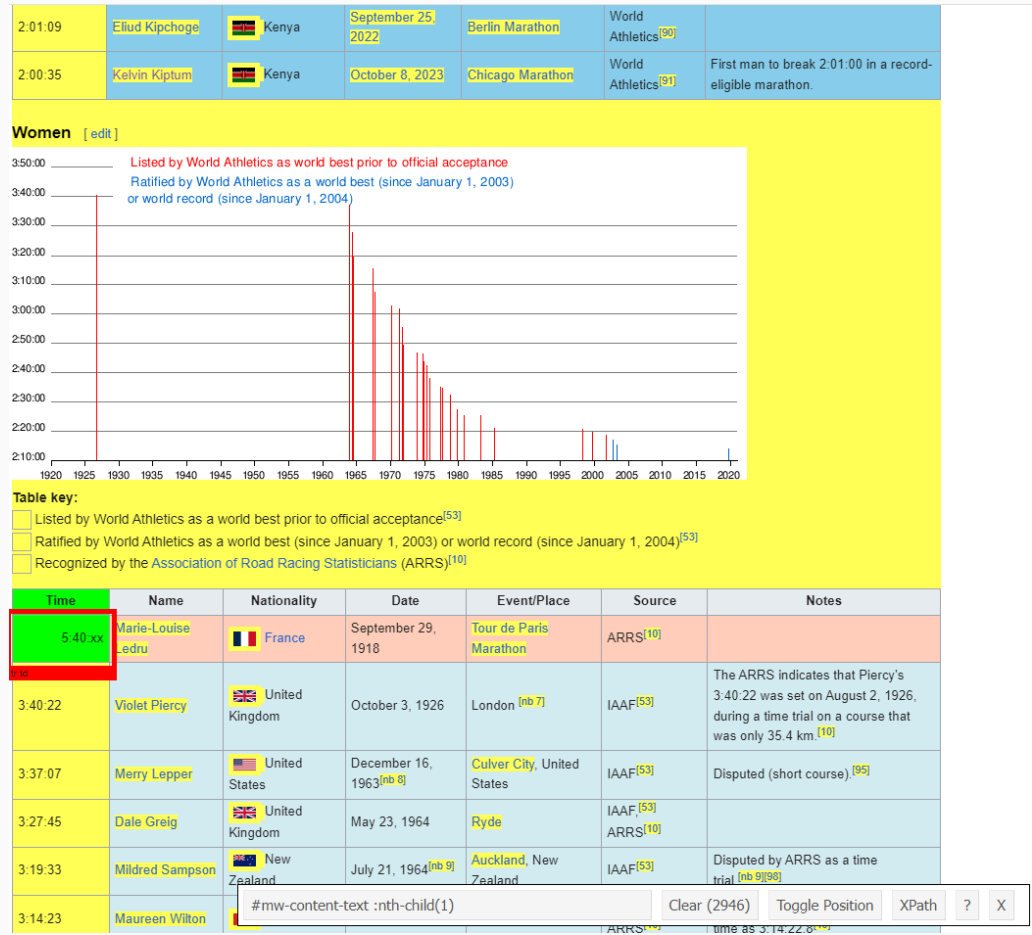
Legend:

- Listed by World Athletics as a world best prior to official acceptance^[53]
- Ratified by World Athletics as a world best (since January 1, 2003) or world record (since January 1, 2004)^[53]
- Recognized by the Association of Road Racing Statisticians (ARRS)^[10]

Time	Name	Nationality	Date	Event/Place	Source	Notes
5:40:xx	Marie-Louise Ledru	 France	September 29, 1918	Tour de Paris Marathon	ARRS ^[10]	
3:40:22	Violet Piercy	 United Kingdom	October 3, 1926	London ^[nb 7]	IAAF ^[53]	The ARRS indicates that Piercy's 3:40:22 was set on August 2, 1926, during a time trial on a course that was only 35.4 km. ^[10]
3:37:07	Merry Lepper	 United States	December 16, 1963 ^[nb 8]	Culver City, United States	IAAF ^[53]	Disputed (short course). ^[95]
3:27:45	Dale Greig	 United Kingdom	May 23, 1964	Ryde	IAAF ^[53] ARRS ^[10]	
3:19:33	Mildred Sampson	 New Zealand	July 21, 1964 ^[nb 9]	Auckland, New Zealand	IAAF ^[53]	Disputed by ARRS as a time trial. ^{[nb 9][96]}
3:14:23	Maureen Wilton	 Canada				

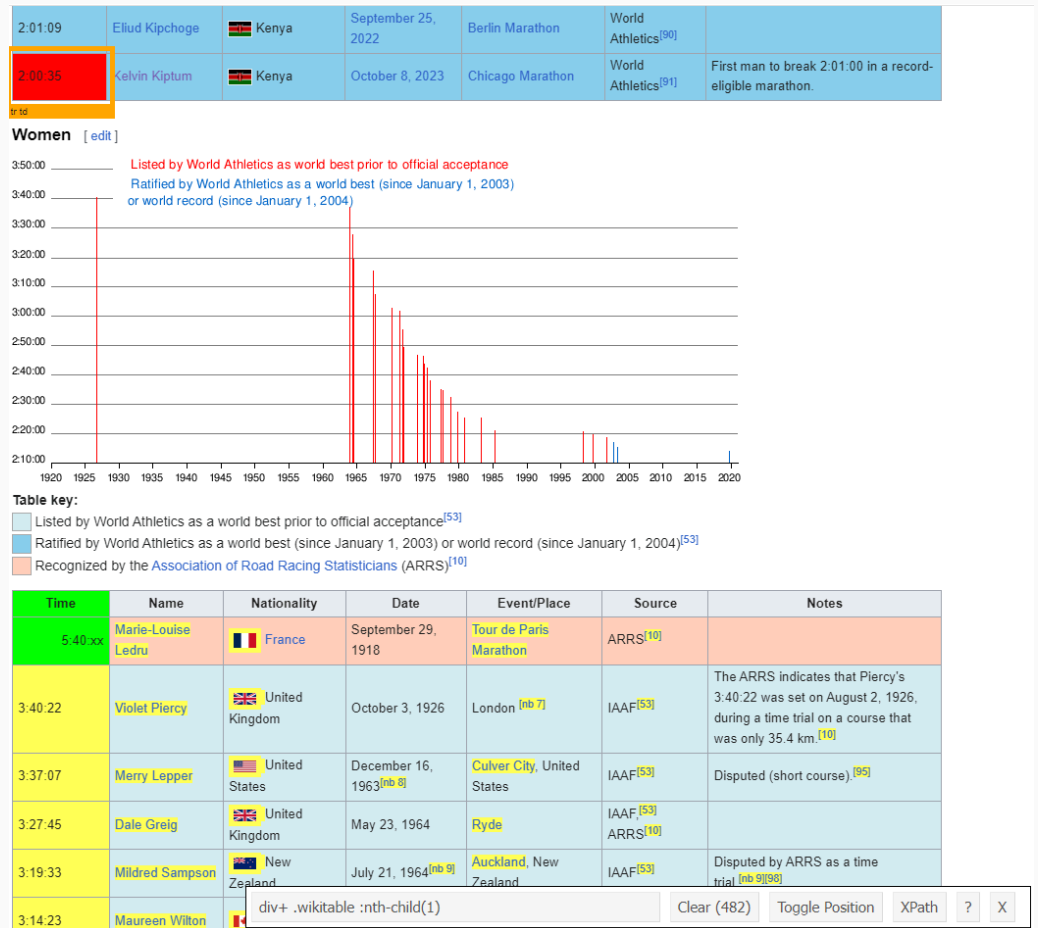
First Table: Women's Records

Clicking on another chart element expands the selection (but too much!)



First Table: Women's Records

Click the elements you
don't want until they
turn red:



First Table: Women's Records

Working through this iterative process yields

```
"div+ .wikitable :nth-child(1)".
```

We can use this unique CSS selector to isolate the women's record table content from the rest of the HTML document.

- **Extract the table content** with `html_element()`
- Parse the **HTML table into an R data frame** with `html_table()`

First Table: Women's Records

- **Extract the table content** with `html_element()`
- Parse the **HTML table into an R data frame** with `html_table()`

```
women <- mthn %>%  
  html_element("div+ .wikitable :nth-child(1)") %>% ## select table  
element  
  html_table() ## convert to data  
frame
```

women

```
## # A tibble: 46 × 7
```

```
##   Time      Name      Nationality    Date    Event/Place
```

```
Source Notes
```

```
##   <chr>      <chr>      <chr>      <chr>    <chr>
```

```
<chr> <chr>
```

```
##   1 5:40:xx Marie-Louise Ledru France      Septe... Tour de Pari...
```

```
ARRS[... " "
```

```
##   2 3:40:22 Violet Piercy      United Kingdom Octob... London[nb 7]
```

```
IAAF[... "The...
```

First Table: Women's Records

Great, it worked! Now convert the date string to a format that R actually understands.²

```
women %>%  
  mutate(Date = mdy(Date)) %>% ## convert string to date format  
  head(2)
```

```
## # A tibble: 2 × 7
```

```
##   Time      Name      Nationality    Date      Event/Place  
Source Notes  
##   <chr>    <chr>          <chr>        <date>    <chr>  
<chr> <chr>  
## 1 5:40:xx Marie-Louise Ledru France      1918-09-29 Tour de Pari...  
ARRS[... "  
## 2 3:40:22 Violet Piercy      United Kingd... 1926-10-03 London[nb 7]  
IAAF[... "The...
```

Note: If column name had spaces or capital letters, we could use the

```
janitor::clean_names()
```

convenience function to clean them. (Q: How else could we have done this?)

First Table: Women's Records

Alright that mostly worked, but there are a few hyperlink references in the dates leading to NAs.

- This is a case where using a regular expression is convenient: match the pattern "[nb X]" at the **end of the strings**

```
women <- women %>%  
  mutate(across(where(is.character), # mutate across all character  
strings  
              ~str_replace(.x, "\\[nb [0-9]+\\]$", "")), # remove [nb  
0-9]  
         Date = mdy(Date)) # convert string to date format  
women
```

```
## # A tibble: 46 × 7
```

```
##   Time      Name      Nationality Date      Event/Place
```

```
Source Notes
```

```
##   <chr>      <chr>      <chr>      <date>      <chr>
```

```
<chr> <chr>
```

Table 2: Men's Records

We could stop here and plot the women's records, but while we're here let's grab the men's records as well.

Challenge: take a couple minutes to use SelectorGadget and find the CSS selector for the men's record table.

- Don't peek at next slide until you give it a shot!

```
men <- mthn %>%  
  html_element("") %>% ## add the selector!  
  html_table() %>%  
  mutate(across(where(is.character), # mutate across all character  
strings  
              ~str_replace(.x, "\\[nb [0-9]+\\]$", "")), # remove [nb  
0-9]  
          Date = mdy(Date))
```

Table 2: Men's Records

```
men ← mthn %>%
  html_element("p+ .wikitable :nth-child(1)") %>% ## add the selector!
  html_table() %>%
  mutate(across(where(is.character), # mutate across all character
strings
           ~str_replace(.x, "\\[nb [0-9]+\\]$", "")), # remove [nb
0-9]
           Date = mdy(Date))
tail(men)
```

```
## # A tibble: 6 × 7
```

##	Time	Name	Nationality	Date	Event/Place	Source
##	<chr>	<chr>	<chr>	<date>	<chr>	<chr>
##	1 2:03:38	Patrick Makau	Kenya	2011-09-25	Berlin Marathon	IAAF,
##	2 2:03:23	Wilson Kipsang	Kenya	2013-09-29	Berlin Marathon	
##	3 2:02:57	Dennis Kimetto	Kenya	2014-09-28	Berlin Marathon	

Browser Inspection Tools

SelectorGadget is a great tool, but sometimes it **takes more work than necessary** and isn't available in all browsers.

Alternate approach: use the **inspect web element**

- Chrome: Right-click > "Inspect" (Ctrl + Shift + I)
- Scroll over source elements until the table of interest is highlighted
- Use the selector that pops up over the element
 - i.e. "table.wikitable"



Duplicate Selectors

If we look at the source code a bit more we can see that the selector `table.wikitable` **isn't unique!**

In cases like this, use `html_elements()` to retrieve all matching elements as a list

```
mthn_tabs ← mthn %>%  
  html_elements("table.wikitable") ## select all elements matching the  
  selector  
  
# first match is men's results  
men2 ← mthn_tabs[[1]] %>% html_table()  
  
# second match is women's results  
women2 ← mthn_tabs[[2]] %>% html_table()
```

Marathon Progression

Challenge: combine both tables into a single dataframe and plot the record progression

- Clean the `Time` variable (get rid of alphanumeric characters at the end)
- Separate the time variable into hours/minutes/seconds using `separate()`
- Use `hours()`, `minutes()`, `seconds()` from `lubridate` to create a time variable for the marathon time
- Create a plot with
 - Marathon time on the y-axis
 - Make sure to include a `scale_y_time()` layer!
 - Date on the x-axis
 - Linetype aesthetic mapping based on the group (men vs. women)
 - Use a nice theme

Answer in a few slides (no peeking until you've tried first)

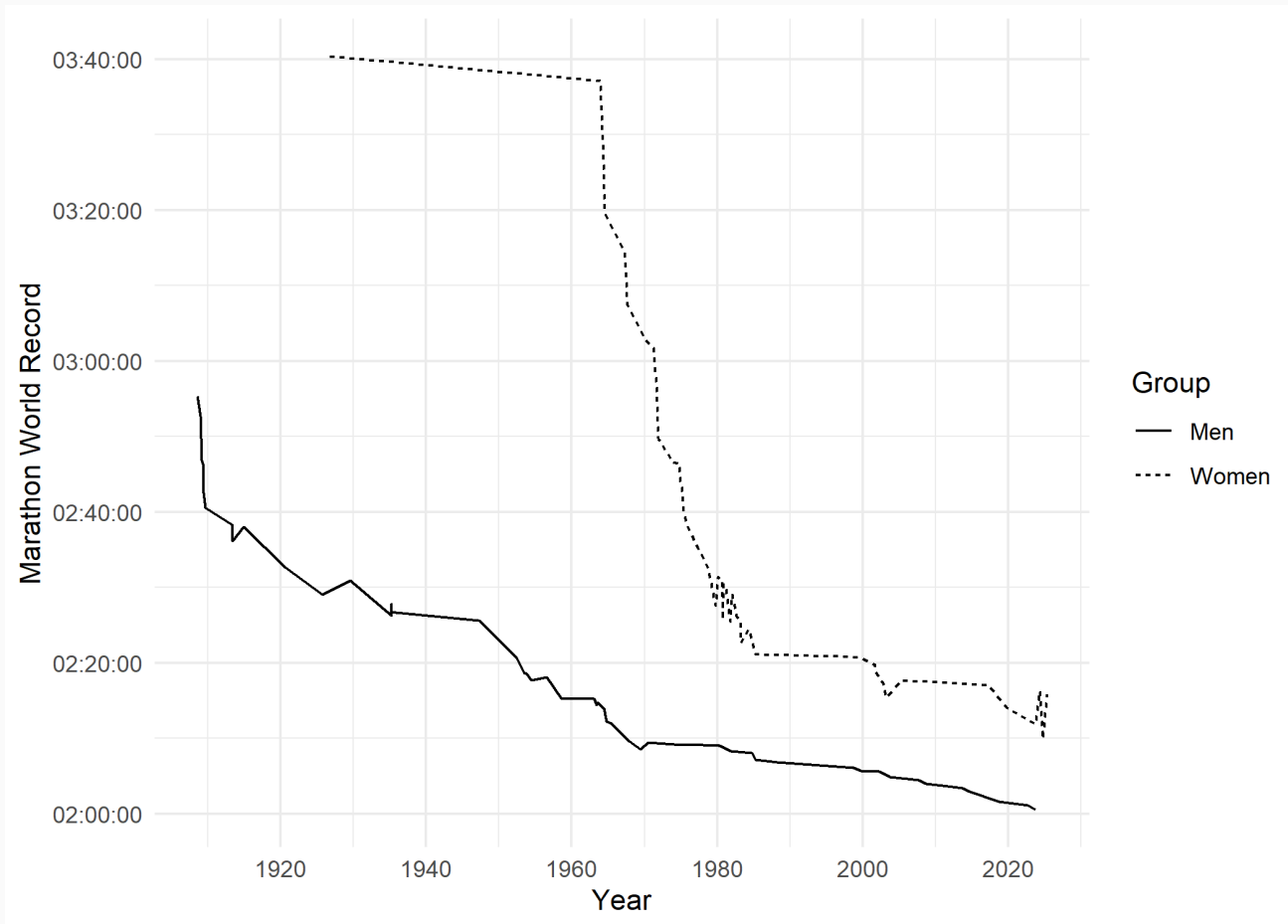
Marathon Progression

```
men ← mutate(men, Group = "Men")
women ← mutate(women, Group = "Women")

mthn_prog ← rbind(men, women) %>%
  mutate(Time = str_replace_all(Time, "[:alpha:]", "")) %>%
  separate(col = "Time", into = c("Hours", "Minutes", "Seconds"), sep =
    ":") %>%
  # add time variable using lubridate functions
  mutate(mthn_time = hours(Hours) + minutes(Minutes) + seconds(Seconds))

ggplot(mthn_prog, aes(x = Date, y = mthn_time)) +
  geom_line(aes(linetype = Group)) +
  scale_y_time() +
  labs(y = "Marathon World Record",
       x = "Year") +
  theme_minimal()
```

Marathon Progression



Another Static Example

Scraping Static Sites

In the last example we saw how to use **rvest** functions to scrape information from **HTML Tables**.

- **Biggest Challenge:** finding the right CSS selectors¹.

Let's work through an example navigating another site's structure to get familiar with more helpful functions.

¹. For more on all the ways to construct CSS selectors, check out [this handy W3 Guide](#)

Example 2: Des Linden

Let's say you got excited by all this marathon talk and wanted to gather more data about one of the most storied American marathoners:\ **Des Linden**

- A native Michigander (originally from Charlevoix, MI)!



Example 2: Des Linden

You know she ran the 2025 Boston Marathon, so you decide to scrape Des' [2025 Boston Marathon results page](#):



129TH BOSTON MARATHON[®]
BANK OF AMERICA

ResultsLeaderboardTrackingMy Runners

Person Details

Participant

Name	Linden, Desiree
Team	—
Bib Number	118
Division	40-44
State/ Province	MI
My Runner	

Result

Place	17
Place (g)	1
Place (Total)	123
Time Total (net)	02:26:19
Time Total (gun)	02:26:19
min/mile	05:35

Race Status

Race State	Finished
Last Split	Finish Net

Splits

Split	Time Of Day	Time	Diff	Min/mile	Miles/h
5K	10:03:58AM	00:16:56	16:56	05:27	11.02
10K	10:20:51AM	00:33:49	16:53	05:26	11.04
15K	10:37:54AM	00:50:52	17:03	05:30	10.93
20K	10:55:00AM	01:07:58	17:06	05:31	10.91
HALF	10:58:46AM	01:11:44	03:46	05:32	10.85
25K	11:12:00AM	01:24:59	13:15	05:28	11.00
30K	11:29:31AM	01:42:29	17:30	05:38	10.65
20 Miles	11:37:18AM	01:50:16	07:47	05:44	10.48
21 Miles	11:43:25AM	01:56:23	06:07	06:07	9.81
35K	11:47:32AM	02:00:30	04:07	05:31	10.91
23 Miles	11:54:50AM	02:07:48	07:18	05:52	10.25
24 Miles	12:00:32PM	02:13:30	05:42	05:42	10.53
40K	12:05:28PM	02:18:27	04:57	05:46	10.44
25.2 Miles	12:07:36PM	02:20:35	02:08	06:12	9.72
Finish Net	12:13:21PM	02:26:19	05:44	05:38	10.66

Features



Results Adjustment Request

Example 2: Boston 2024 Results

To start, read in the page's HTML.

```
des_25 ← read_html("https://results.baa.org/2025/?  
content=detail&fpid=search&pid=search&idp=9TGHS6FF1AF9F5&lang=EN_CAP&even·
```


Example 2: Boston 2024 Results

What do we see when we look at the page?

des_25

```
## {html_document}
## <html lang="en">
## [1] <head>\n<title>Boston Athletic Association</title>\n<meta http-equiv="Con
...
## [2] <body class="pid-search content-detail view-public branch-production site
...
```

1. The page's head information (`<head>`)
 - Contains metadata about the page (not actually displayed)
2. The page's body contents (`<body>`)
 - This is where all the stuff is

Example 2: Boston 2025 Results

Look at the body's **child elements** with `html_children()`

- all 10 elements nested under `body`

```
html_element(des_25, "body") %>% html_children()

## {xml_nodeset (10)}
## [1] <header class="hidden-xs"><div class="container">\n<div
class="logo row" ...
## [2] <nav class="navbar navbar-default navbar-top navbar-fixed-top">
<div clas ...
## [3] <script async src="https://www.googletagmanager.com/gtag/js">
</script>
## [4] <script>\r\n          \r\n          var mt_tracker = {}; \r\n
\r\n    ...
## [5] <section class="section-nav container"><nav class="navbar navbar-
default ...
## [6] <section class="section-main"><div class="container" id="main-
content">\ ...
## [7] <footer><div class="container">\n<div class="row">\n<div
```

Example 2: Boston 2025 Results

Look at the body's **child elements** with `html_children()`

- all 10 elements nested under `body`

This matches the **page source** (access with "Inspect")

```
<body class="pid-search content-detail view-public branch-production site-2025_boston event  
-R"> == $0  
  <header class="hidden-xs">...</header>  
  <nav class="navbar navbar-default navbar-top navbar-fixed-top">...</nav>  
  <!-- header end -->  
  <!-- main start -->  
  <script async src="https://www.googletagmanager.com/gtag/js"></script>  
  <script>...</script>  
  <section class="section-nav container">...</section>  
  <section class="section-main">...</section>  
  <!-- section-main -->  
  <!-- main end -->  
  <!-- footer start -->  
  <footer>...</footer>  
  <!-- footer section end -->  
  <div class="popup-info" style="display:none;" id="popup-info"></div>  
  <div class="popup" style="display:none;" id="popup"></div>  
  <div id="cconsent-overlay" style="display:none;"></div>
```

Example 2: Boston 2025 Results

Let's start by grabbing all the information from the left half of the page.

Using SelectorGadget, identify the selector for the left-side contents.

Person Details

Participant

Name	Linden, Desiree
Team	-
Bib Number	118
Division	40-44
State/ Province	MI
My Runner	

Result

Place	17
Place (g)	1
Place (Total)	123
Time Total (net)	02:26:19
Time Total (gun)	02:26:19
min/mile	05:35

Race Status

Race State	Finished
Last Split	Finish Net

Splits

Split	Time Of Day	Time	Diff	Min/mile	Miles/h
5K	10:03:58AM	00:16:56	16:56	05:27	11.02
10K	10:20:51AM	00:33:49	16:53	05:26	11.04
15K	10:37:54AM	00:50:52	17:03	05:30	10.93
20K	10:55:00AM	01:07:58	17:06	05:31	10.91
HALF	10:58:46AM	01:11:44	03:46	05:32	10.85
25K	11:12:00AM	01:24:59	13:15	05:28	11.00
30K	11:29:31AM	01:42:29	17:30	05:38	10.65
20 Miles	11:37:18AM	01:50:16	07:47	05:44	10.48
21 Miles	11:43:25AM	01:56:23	06:07	06:07	9.81
35K	11:47:32AM	02:00:30	04:07	05:31	10.91
23 Miles	11:54:50AM	02:07:48	07:18	05:52	10.25

Example 2: Boston 2025 Results

```
des_25_l ← html_element(des_25, ".channel-left") %>%  
  html_element("div.untabbed-boxes")
```

What's here?

```
des_25_l
```

```
## {html_node}  
## <div class="untabbed-boxes">  
## [1] <div class="detail-box box-general" id="detail-box-general"  
role=" ..." ...  
## [2] <div class="detail-box box-totals" id="detail-box-totals"  
role=""> ...
```

Two elements:

1. detail-box box-general: class of top half div
2. detail-box box-totals: class of bottom half div

Example 2: Boston 2025 Results

Start by grabbing the top half info with the full class:

```
des_25_lu ← html_element(des_25_l, ".detail-box.box-general")
```

Q: did we need the first half (`detail-box`)?

```
identical(des_25_lu,  
          html_element(des_25_l, ".box-general")  
          )
```

```
## [1] TRUE
```

A: No! since there's only one `box-general` here we can refer to it directly.

Elements vs. Element

We can see the difference with `html_element()` and `html_elements()`

- `html_element()` retrieves the **HTML nodes** directly
 - same length as the input
 - If a single node, just one thing gets returned
 - if a nodeset, returns a same-length vector
- `html_elements()` retrieves the **nodeset itself**
 - Contains all nodes that match

```
html_element(des_25, ".detail-  
box")
```

```
## {html_node}  
## <div class="detail-box box-  
general" id="detail-box-  
general" role="">
```

```
html_elements(des_25, ".detail-  
box")
```

```
## {xml_nodeset (5)}  
## [1] <div class="detail-box  
box-general" id="detail-box-  
general" role=" ...  
## [2] <div class="detail-box  
box-totals" id="detail-box-  
totals" role=""> ...  
## [3] <div class="detail-box
```

Example 2: Boston 2025 Results

What does the left upper box contain?

```
des_25_lu
```

```
## {html_node}  
## <div class="detail-box box-general" id="detail-box-general"  
role="">  
## [1] <h3>Participant\n</h3>  
## [2] <table class="table table-condensed"><tbody>\n<tr class=" f-  
__fullname">\ ...
```

What we grabbed *itself* contains two elements:

1. An `h3` title, and
2. a table (`table`)

Example 2: Boston 2025 Results

Alternatively, to view **all elements AND all child elements**, use the wildcard selector "*" in `html_elements()`

- Ignores structure/nesting

```
html_elements(des_25_lu, "*")
```

```
## {xml_nodeset (24)}
## [1] <h3>Participant\n</h3>
## [2] <table class="table table-condensed"><tbody>\n<tr class=" f-__fullname">
...
## [3] <tbody>\n<tr class=" f-__fullname">\n<th class="desc">Name</th>\n<td cla
...
## [4] <tr class=" f-__fullname">\n<th class="desc">Name</th>\n<td class="f-__f
...
## [5] <th class="desc">Name</th>
## [6] <td class="f-__fullname last">Linden, Desiree</td>
## [7] <tr class="list-highlight f-team">\n<th class="desc">Team</th>\n<td clas
...
## [8] <th class="desc">Team</th>
## [9] <td class="f-team last"><span class="text-muted">-\n</span></td>
## [10] <span class="text-muted">-\n</span>
```

Grabbing Text

If we want to grab the text from the third-level heading `h3`, we can use

1. `html_text()`: retrieves the **raw, underlying text**
2. `html_text2()`: retrieves the text **as displayed in a browser**

```
html_elements(des_25_lu, "h3")
%>%
  html_text()
```

```
## [1] "Participant\n"
```

```
html_elements(des_25_lu, "h3")
%>%
  html_text2()
```

```
## [1] "Participant"
```

Here the difference is whether we keep the line break special character `\n` or not.

Example 2: Boston 2025 Results

Grabb the (full) table with `html_table()`

- `html_table()` recognizes the tabular format of an HTML table, automatically converts to a tibble data frame (easiest option)

```
part_tab ← des_25_lu %>%  
  html_element("table.table-condensed") %>%  
  html_table()
```

```
part_tab
```

```
## # A tibble: 6 × 2  
##   X1          X2  
##   <chr>      <chr>  
## 1 Name      "Linden, Desiree"  
## 2 Team      "-"  
## 3 Bib Number "118"  
## 4 Division  "40-44"  
## 5 State/ Province "MI"  
## 6 My Runner  ""
```

Example 2: Boston 2025 Results

Challenge: get the contents of the "Result" table (bottom-left)

Workflow:

- Start with `des_25_1`
- Use `html_element()` (likely twice) to get the bottom table
- Format as a data frame with `html_table()`

129TH BOSTON MARATHON
BANK OF AMERICA

Results Leaderboard Tracking My Runners

Person Details

Participant

Name	Linden, Desiree
Team	-
Bib Number	118
Division	40-44
State/ Province	MI
My Runner	+

Race Status

Race State	Finished
Last Split	Finish Net

Splits

Split	Time Of Day	Time	Diff	Min/mile	Miles/h
6K	10:03:58AM	00:16:56	16:56	05:27	11.02
10K	10:20:51AM	00:33:49	16:53	05:26	11.04
15K	10:37:54AM	00:50:52	17:03	05:30	10.93
20K	10:55:00AM	01:07:58	17:06	05:31	10.91
HALF	10:58:46AM	01:11:44	03:46	05:32	10.85
25K	11:12:00AM	01:24:59	13:15	05:28	11.00
30K	11:29:31AM	01:42:29	17:30	05:38	10.65
20 Miles	11:37:18AM	01:50:16	07:47	05:44	10.48
21 Miles	11:43:25AM	01:56:23	06:07	06:07	9.81
35K	11:47:32AM	02:00:30	04:07	05:31	10.91
23 Miles	11:54:50AM	02:07:48	07:18	05:52	10.25
24 Miles	12:00:32PM	02:13:30	05:42	05:42	10.53
40K	12:05:28PM	02:18:27	04:57	05:46	10.44
25.2 Miles	12:07:36PM	02:20:35	02:08	06:12	9.72
Finish Net	12:13:21PM	02:26:19	05:44	05:38	10.66

Result

Place	17
Place (g)	1
Place (Total)	123
Time Total (net)	02:26:19
Time Total (gun)	02:26:19
min/mile	05:35

Features

Results Adjustment Request

Marathonfoto.com

Example 2: Boston 2025 Results

```
res_tab <- des_25_l %>%  
  html_element(".box-totals") %>%  
  html_element(".table-condensed") %>%  
  html_table()  
head(res_tab)
```

```
## # A tibble: 6 × 2  
##   X1                X2  
##   <chr>            <chr>  
## 1 Place            17  
## 2 Place (g)        1  
## 3 Place (Total)    123  
## 4 Time Total (net) 02:26:19  
## 5 Time Total (gun) 02:26:19  
## 6 min/mile         05:35
```

Example 2: Boston 2025 Results

Now that we have the left side stuff, let's grab the right side!

Conveniently, the selector follows the same naming convention

```
des_25_r ← des_25 %>% html_element(".channel-right") %>%  
  html_element("div.untabbed-boxes")  
des_25_r
```

```
## {html_node}  
## <div class="untabbed-boxes">  
## [1] <div class="detail-box box-state" id="detail-box-state"  
role="">\n ...  
## [2] <div class="detail-box box-splits" id="detail-box-splits"  
role=""> ...  
## [3] <div class="detail-box box-features" id="detail-box-  
features" role ...
```

Example 2: Boston 2025 Results

Here we can see that all the right-side info is *also* being stored in HTML tables.

We *could* manually grab both the tables again... *or* we can take advantage of `html_elements()` retrieving all matches at once

- Pull *all* `table` elements from `des_25`

```
html_elements(des_25, ".table")
```

```
## {xml_node} (5)
## [1] <table class="table table-condensed"><tbody>\n<tr class=" f-
__fullname">\ ...
## [2] <table class="table table-condensed"><tbody>\n<tr class=" f-
place_all">\n ...
## [3] <table class="table table-condensed"><tbody>\n<tr class=" f-
race_status"> ...
## [4] <table class="table table-condensed table-striped">\n<thead>
<tr>\n<th cla ...
```

Example 2: Boston 2025 Results

Finally, let's get the url links from the "Features" box buttons

**129TH BOSTON MARATHON[®]**
BANK OF AMERICA

ResultsLeaderboardTrackingMy Runners

Person Details

Participant

Name	Linden, Desiree
Team	—
Bib Number	118
Division	40-44
State/ Province	MI
My Runner	

Result

Place	17
Place (g)	1
Place (Total)	123
Time Total (net)	02:26:19
Time Total (gun)	02:26:19
min/mile	05:35

Race Status

Race State	Finished
Last Split	Finish Net

Splits

Split	Time Of Day	Time	Diff	Min/mile	Miles/h
5K	10:03:58AM	00:16:56	16:56	05:27	11.02
10K	10:20:51AM	00:33:49	16:53	05:26	11.04
15K	10:37:54AM	00:50:52	17:03	05:30	10.93
20K	10:55:00AM	01:07:58	17:06	05:31	10.91
HALF	10:58:46AM	01:11:44	03:46	05:32	10.85
25K	11:12:00AM	01:24:59	13:15	05:28	11.00
30K	11:29:31AM	01:42:29	17:30	05:38	10.65
20 Miles	11:37:18AM	01:50:16	07:47	05:44	10.48
21 Miles	11:43:25AM	01:56:23	06:07	06:07	9.81
35K	11:47:32AM	02:00:30	04:07	05:31	10.91
23 Miles	11:54:50AM	02:07:48	07:18	05:52	10.25
24 Miles	12:00:32PM	02:13:30	06:42	05:42	10.53
40K	12:05:28PM	02:18:27	04:57	05:46	10.44
25.2 Miles	12:07:36PM	02:20:35	02:08	06:12	9.72
Finish Net	12:13:21PM	02:26:19	06:44	05:38	10.66

Features

[Results Adjustment Request](#)

Marathonfoto.com

Retrieving Links

Workflow:

- Start with `des_25`

```
des_25
```

```
## {html_document}  
## <html lang="en">  
## [1] <head>\n<title>Boston  
Athletic  
Association</title>\n<meta http-  
equiv="Con ...  
## [2] <body class="pid-search  
content-detail view-public branch-  
production site ...
```

Retrieving Links

Workflow:

- Start with `des_25`
- Navigate to the right side
(`.channel-right`)

```
des_25 %>%
```

```
  html_element(".channel-right")
```

```
## {html_node}  
## <div class="col col-xs-12 col-  
md-6 detail-channel channel-  
right">  
## [1] <div class="untabbed-  
boxes">\n<div class="detail-box  
box-state      " id ...
```

Retrieving Links

Workflow:

- Start with `des_25`
- Navigate to the right side
(`.channel-right`)
- Navigate into the feature box
(`.box-features`)

```
des_25 %>%  
  html_element(".channel-right")  
%>%  
  html_element(".box-features")
```

```
## {html_node}  
## <div class="detail-box box-  
features      " id="detail-box-  
features" role="">  
## [1] <h3>Features\n</h3>  
## [2] <table class="table table-  
condensed"><tbody><tr>\n<td  
class=" last"><div  ...
```

Retrieving Links

Workflow:

- Start with `des_25`
- Navigate to the right side
(`.channel-right`)
- Navigate into the feature box
(`.box-features`)
- Retrieve the elements matching the button class (`a.btn.btn-primary.btn-block`)

```
des_25 %>%  
  html_element(".channel-right")  
%>%  
  html_element(".box-features")  
%>%  
  html_elements("a.btn.btn-  
primary.btn-block")  
  
## {xml_nodeset (2)}  
## [1] <a class="btn btn-primary  
btn-block"  
href="https://www.marathonfoto.com/I  
...  
## [2] <a class="btn btn-primary  
btn-block"  
href="https://www.baa.org/boston-  
mar ...
```

Retrieving Links

Workflow:

- Start with `des_25`
- Navigate to the right side (`.channel-right`)
- Navigate into the feature box (`.box-features`)
- Retrieve the elements matching the button class (`a.btn.btn-primary.btn-block`)
- Use `html_attr("href")` to retrieve all values of the `href` attributes

```
des_25 %>%  
  html_element(".channel-right")  
%>%  
  html_element(".box-features")  
%>%  
  html_elements("a.btn.btn-  
primary.btn-block") %>%  
  html_attr("href")  
  
## [1]  
"https://www.marathonfoto.com/In?  
&RaceOID=12902025S1&LastName=Linden&Bib=  
## [2]  
"https://www.baa.org/boston-  
marathon-results-change-request?  
bib=118&fname=Desiree&lname=Linden"
```

Structured v Unstructured

Here nearly everything was nicely formatted in HTML tables, which `html_table()` automatically converts into a structured format.

If what you want is unstructured/not together, you'll need to figure out the **individual selectors** and put them together in a **custom data object**

```
data.frame(  
  athlete = "Des Linden",  
  title = html_element(des_25, "a.navbar-brand") %>%  
    html_element(".hidden-xs") %>% html_text2(),  
  back_link = html_element(des_25, ".tt-help") %>% html_attr("href")  
)
```

```
##           athlete                               title  
## 1 Des Linden Boston Athletic Association | 2025  
##  
back_link  
## 1 /2025/?
```

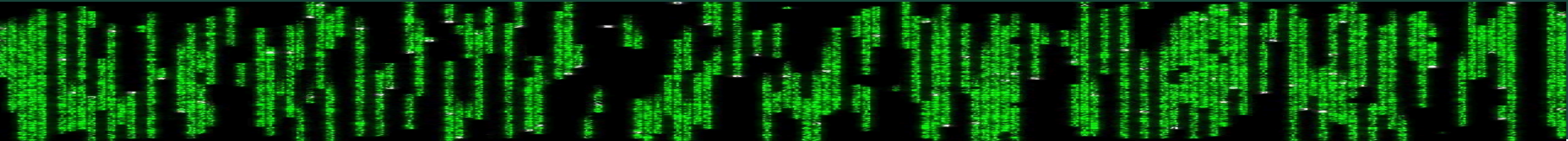
```
event=R&event main group=runner&pid=start&search%5Bname%5D=linden
```

Summary: Static Scraping

When scraping static web content rendered server-side:

- Start by finding the relevant CSS selectors
- Use the **rvest** package to read in the HTML document and parse it
 - Tabular workflow: `read_html(URL) %>%
html_elements(CSS_Selectors) %>% html_table()`
 - Might need other functions depending on content type (e.g.
`html_text/text2()`, `html_attr()`, `html_children()`)

Interacting with Static Sites



While scraping a **known url** is pretty cool, we can do more than that.

We already saw one way to progress through a url chain, but we can **go deeper**.

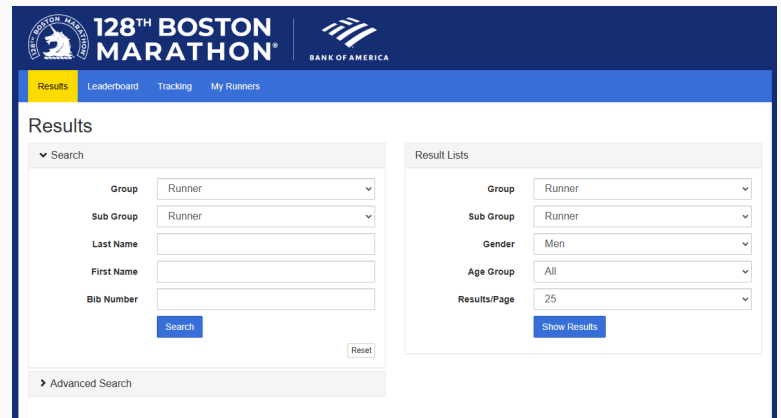
We can go **into.the.matrix**.

Interacting with Static Sites

Let's go back to the beginning with our Boston Marathon 2024 workflow.

Suppose rather than just Des' results, you wanted to

1. Navigate back to the main results search page
 1. Enter custom search term (i.e. specific age group/gender) and execute a search from the home page
 2. Retrieve all results from the first results page
 3. Navigate to the next page and repeat the process



The screenshot shows the '128th BOSTON MARATHON' website with the 'BANK OF AMERICA' logo. The navigation bar includes 'Results', 'Leaderboard', 'Tracking', and 'My Runners'. The 'Results' section is active, displaying a search form on the left and a 'Result Lists' section on the right. The search form includes fields for 'Group' (Runner), 'Sub Group' (Runner), 'Last Name', 'First Name', and 'Bib Number', with a 'Search' button and a 'Reset' link. The 'Result Lists' section includes fields for 'Group' (Runner), 'Sub Group' (Runner), 'Gender' (Men), 'Age Group' (All), and 'Results/Page' (25), with a 'Show Results' button. An 'Advanced Search' link is also visible at the bottom of the search form.

We can do this with **rvest**.

Interacting with Web Forms

To **interact with web forms**, we'll use the below workflow:

- Retrieve a target form with `html_form()`
- Fill in the form's required elements with `html_form_set()`
- Submit the form with `html_form_submit()`
- Read the resulting url with `html_read()`.

Let's practice this by trying to retrieve the top 50 results for my age group (Men 18-39)

Interacting with Web Forms

Start by navigating to the [Boston Marathon Results Search](#) (the main search page).

128TH BOSTON MARATHON | BANK OF AMERICA

Results | Leaderboard | Tracking | My Runners

Results

▼ Search

Group Runner ▼

Sub Group Runner ▼

Last Name

First Name

Bib Number

Search

Reset

► Advanced Search

Result Lists

Group Runner ▼

Sub Group Runner ▼

Gender Men ▼

Age Group All ▼

Results/Page 25 ▼

Show Results

Interacting with Web Forms

Here, there are **multiple forms** with the same class (`form-horizontal`) but different names

- `form_search_simple`: the top-left "Search"
- `form_search_advanced`: the bottom-left "Advanced Search"
- `form_lists_default`: the top-right "Results List" search

```
bos_srch ← read_html("https://results.baa.org/2025/?
pid=search&pidp=start")
html_form(bos_srch)

## [[1]]
## <form> 'form_search_simple' (POST https://results.baa.org/2025/?
pid=search&pidp=start)
##   <field> (hidden) lang: EN_CAP
##   <field> (hidden) startpage: start_responsive
##   <field> (hidden) startpage_type: search
##   <field> (select) event_main_group: runner
##   <field> (select) event:
```

Interacting with Web Forms

Use `html_form()` to extract the form we want (Results List):

To do this, we could use the `class[attr = \"value\"]` selector syntax to retrieve it as an element by name using our previous workflow

```
res_form ← html_elements(bos_srch, "form[name =  
\"form_lists_default\"]") %>%  
  html_form()  
class(res_form)
```

```
## [1] "list"
```

But that nests the form within a list.

Interacting with Web Forms

Use `html_form()` to extract the form we want (Results List):

Alternatively, just run `html_form()` on the main page and select the list item we want using list indexing

```
html_form(bos_srch)[[3]]
```

```
## <form> 'form_lists_default' (POST https://results.baa.org/2025/?  
pid=list&pidp=start)  
##   <field> (hidden) lang: EN_CAP  
##   <field> (hidden) startpage: start_responsive  
##   <field> (hidden) startpage_type: lists  
##   <field> (select) event_main_group: runner  
##   <field> (select) event:  
##   <field> (text) search[sex]:  
##   <field> (text) search[age_class]:  
##   <field> (select) num_results: 25  
##   <field> (button) submit:
```

Interacting with Web Forms

Every web form has several different **fields** we'll need to fill or interact with

Here we've got:

- 3 **hidden** fields that **we can't interact with**
- 3 **select** fields that have **dropdown menus**
- 2 **text subfields**: "search[sex]" and "search[age_class]" in the `event` dropdown
- 1 **button** named "submit" we'd click when manually searching

Filling Web Forms with *html_form_set()*

Let's set up our search by setting

- Age Group to 19-39
- 50 Results/Page

```
search_set <- html_form_set(res_form[[1]],  
                             "event" = "runner",  
                             "search[sex]" = "M",  
                             "search[age_class]" = "18-39",  
                             "num_results" = 50)
```

Submitting Web Forms

Submitting the form and getting the response:

```
resp ← html_form_submit(search_set) # submit the form
resp

## Response [https://results.baa.org/2025/?pid=list&pidp=start]
##   Date: 2025-09-29 19:46
##   Status: 200
##   Content-Type: text/html; charset=UTF-8
##   Size: 20.8 kB
## <!DOCTYPE html>
## <html lang="en">
## <head>
## <title>Boston Athletic Association</title>
## <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
## <meta name="apple-mobile-web-app-status-bar-style" content="black" />
## <link rel="apple-touch-icon" href="//results-
static.mikatiming.com/2025/bosto ...
## <meta name="viewport" content="width=device-width, initial-scale=1">
## <link rel="stylesheet" href="//results-
```

Web Form Response

Submitting the form gets us a `response` class object, containing some useful components, including the resulting url:

```
resp$url
```

```
## [1] "https://results.baa.org/2025/?pid=list&pidp=start"
```

And we can read the response url as we usually do and navigate to the results element we want:

```
# get the result url's html  
res ← read_html(resp) %>% html_elements("div.col-sm-12.row-xs") %>%  
html_elements("li")
```

Web Form Response

And looking at our results:

```
res[2] %>% html_text()
```

```
## [1] "\n\nThere are currently no results available.\n\n"
```

Ahh dangit. Turns out that this page is **client-side** rendered

- The page's content is **dynamically-generated** based on our search.

In order to scrape a client-side site, we're going to need more tools...

Table of Contents

1. **Prologue**
2. **Intro to Web Scraping**
3. **Scraping Static Websites**